

Connect Four

AI Minimax and Alpha-Beta Pruning



Suatu hari Mbappe dan Vinicius sedang bermain game. Nama gamenya adalah Connect Four. Namun, karena Mbappe dengan sifat diktatornya bosan, dia ingin mengembangkan game ini supaya ada AI. Dia menyuruh Vinicius terlebih dahulu untuk merancang spesifikasi proyek ini karna gak mungkin diktaktor kerjain sendiri, dia pasti nyuruh orang.

1. Tujuan Permainan

Connect Four adalah permainan strategi dua pemain pada papan tegak berukuran **7 kolom × 6 baris**. Tujuan setiap pemain adalah menjadi yang pertama menyusun **empat bidak miliknya secara sejajar**, baik horizontal, vertikal, maupun diagonal sekaligus menghalangi lawan mencapai hal yang sama. Bila seluruh papan terisi tanpa ada yang berhasil, permainan berakhir **seri**.

Pada implementasi ini, pemain manusia (**X**) dapat bertanding melawan komputer (**O**), melawan pemain manusia lain, atau menyaksikan dua AI bertanding. Komputer menentukan langkahnya menggunakan algoritma pencarian **minimax dengan alpha-beta pruning** dalam formulasi **negamax**, dilengkapi **Transposition Table**, **statistik pencarian**, empat **tingkat kesulitan** (termasuk Expert dengan **Opening Book**), serta empat **gaya bermain (personality)**.

Di luar bermain langsung, program menyediakan perkakas analisis (Explainable AI, Position Analysis, Heat Map, Search Tree Visualization, Analisis Kompleksitas, Zobrist Hash, Principal Variation), mode kompetisi (AI vs AI Tournament, Benchmark), serta mode latihan (Puzzle Mode) dan produktivitas (Undo/Redo tak terbatas, Replay interaktif dengan ekspor ASCII/JSON, saran Adaptive AI).

1.1 Ringkasan Fitur Wajib

- **Transposition Table** : cache nilai evaluasi *exact* per langkah AI.
- **AI Statistics** : Depth, Nodes, Alpha/Beta Cutoff, Time tiap AI berpikir.
- **Search Tree Visualization** (tree.) : pohon 2-ply dengan alpha-beta nyata.
- **Explainable AI** (explain.) : alasan di balik langkah AI + skor evaluasi.

- **Opening Book** : buku pembuka ilustratif, dicek sebelum minimax.
- **4 Tingkat Kesulitan** : Mudah, Sedang, Sulit, **Expert** (+ Custom).
- **AI vs AI Tournament** : N pertandingan otomatis, sisi ditukar tiap match genap.
- **Benchmark AI** : tabel Depth/Nodes/Time kedalaman 1–5.
- **Heat Map** (heatmap.) : evaluasi seluruh 7 kolom sekaligus.
- **Undo & Redo tak terbatas** : riwayat penuh dua arah.

1.2 Ringkasan Fitur Bonus

- **Replay Interaktif** (next./prev./jump(N).) : navigasi bebas riwayat.
- **Export Replay** : ASCII animation dan JSON.
- **Position Analysis** (analysis.) : Center Control, Threat, Connect-2/3, Total.
- **AI Personality** : Balanced / Aggressive / Defensive / Random.
- **Smart Hint** : konfirmasi (y/n) sebelum langkah berisiko.
- **Zobrist Hash** (zobrist.) : hash posisi, didemonstrasikan mandiri.
- **Principal Variation** (pv.) : garis terbaik hasil pencarian.
- **Adaptive AI** : saran penyesuaian kesulitan dari riwayat menang/kalah.
- **Analisis Kompleksitas** (complexity.) : branching factor, estimasi node, pruning efficiency.
- **Puzzle Mode** : 5 puzzle, jawaban diverifikasi programatik.

2. Deskripsi Permainan

Kedua pemain bergiliran menjatuhkan bidak ke salah satu dari tujuh kolom. Karena pengaruh gravitasi, bidak selalu menempati **sel kosong terbawah** pada kolom yang dipilih. Papan direpresentasikan sebagai **list berisi 7 kolom**, dan setiap kolom adalah **list berisi 6 sel** terurut dari bawah ke atas. Setiap sel bernilai:

- **x** : bidak pemain manusia,
- **o** : bidak komputer (AI),
- **e** : sel kosong (empty).

Contoh papan awal (seluruhnya kosong) dan sebuah papan tengah permainan lengkap dengan penanda langkah terakhir, ditampilkan program sebagai berikut:

```

| . . . . . . |   | . . . O . . . |
| . . . . . . |   | . . O X . . . |
| . . . . . . |   | . . X X O . . |
| . . . . . . |   | . . O O X . . |
| . . . . . . |   | . O X O X . . |
| . . . . . . |   | . O X X O . X |
-----
1 2 3 4 5 6 7     1 2 3 4 5 6 7
                    ^ (terakhir)

```

Simbol: **X** = pemain, **O** = AI, **.** = kosong. Tanda ^ di bawah papan menunjukkan kolom langkah terakhir.

2.1 Representasi Internal Tambahan

Selain papan, program menyimpan sejumlah *dynamic predicate* untuk mendukung fitur lanjutan — semuanya dideklarasikan terpusat di awal berkas agar mudah ditelusuri:

Predicate	Penjelasan
game_state(B, Turn, Depth)	State permainan aktif: papan, giliran, kedalaman AI.
undo_stack/1, redo_stack/1	List snapshot s(Board, Turn, Depth, MoveLog) untuk undo/redo.

<code>move_log/1</code>	Riwayat kolom yang sudah dimainkan,urut dari awal.
<code>tt_entry(B,P,D,V)</code>	Entri Transposition Table: nilai <i>exact</i> pada posisi/giliran/kedalaman tertentu.
<code>search_counter(Key,N)</code>	Statistik pencarian aktif: nodes, alpha_cut, beta_cut, tt_hit, tt_miss.
<code>book_move_fact(Log,Col)</code>	Entri Opening Book, dikunci oleh prefiks move_log.
<code>zobrist_val(C,R,P,V)</code>	Tabel nilai acak Zobrist per (kolom, baris, bidak).
<code>puzzle_fact(Id,B,Turn,Ds)</code>	Bank posisi Puzzle Mode.
<code>stat(Key,N)</code>	Statistik permanen menang/kalah/seri, tersimpan ke stats.txt.

3. Alur Permainan

1. Program menampilkan menu utama berisi 11 opsi: empat mode bermain (Kamu vs AI, AI vs Kamu, Dua Pemain, Demo AI vs AI), Muat permainan, Replay interaktif, Statistik, Benchmark AI, AI vs AI Tournament, Puzzle Mode, dan Keluar.
2. Untuk mode melawan AI (1/2/4), pemain memilih **tingkat kesulitan** (Mudah / Sedang / Sulit / Expert / Custom) dan **gaya bermain AI** (Balanced / Aggressive / Defensive / Random). Sebelum bertanya kedalaman, program menampilkan **saran Adaptive AI** bila riwayat menang/kalah cukup banyak dan berat sebelah.
3. Papan kosong ditampilkan. Pemain **X** jalan lebih dulu (kecuali mode “AI vs Kamu”, di mana AI sebagai X jalan duluan).
4. Setiap giliran manusia, program memeriksa **ancaman lawan** dan menampilkan peringatan bila lawan bisa menang giliran berikutnya.
5. Pemain mengetik nomor kolom (1–7) atau salah satu dari belasan perintah analisis/utilitas (lihat Bagian 5). Bila langkah yang dipilih justru **membuka kemenangan lawan**, **Smart Hint** meminta konfirmasi (y/n) sebelum benar-benar dijalankan.
6. Langkah divalidasi: kolom penuh atau di luar jangkauan **ditolak** dan pemain diminta mengulang.
7. Giliran AI: memilih opening book bila posisi cocok (khusus Expert), atau menjalankan negamax dengan alpha-beta + Transposition Table pada kedalaman dan gaya yang dipilih, lalu menampilkan statistik pencarian.
8. Setelah setiap langkah, program memeriksa kondisi akhir (menang 4-sejajar atau papan penuh). Bila belum berakhir, giliran berganti dan siklus berulang.
9. Saat permainan berakhir, statistik menang/kalah/seri diperbarui dan disimpan permanen ke stats.txt.

4. Aturan Permainan

1. Papan berukuran 7 kolom (bernomor 1–7) dan 6 baris.
2. Dua pemain bergiliran; pemain **X** jalan pertama kecuali pada mode “AI (X) vs Kamu (O)”.
3. Satu langkah = menjatuhkan satu bidak ke sebuah kolom; bidak mengisi sel kosong terbawah kolom tersebut.
4. Kolom yang sudah penuh (6 bidak) tidak boleh dipilih.
5. Pemain menang bila memiliki **4 bidak berurutan** pada satu garis: horizontal, vertikal, atau diagonal (naik maupun turun).

6. Bila 42 sel terisi penuh tanpa garis 4-sejajar, permainan **seri**.
7. Undo membatalkan satu *giliran manusia* beserta balasan AI-nya sekaligus (bila melawan AI); Redo mengembalikannya. Keduanya dapat dilakukan berulang kali (tak terbatas), selama masih ada riwayat.

5. Daftar Predicate & Command Utama

Catatan : Predicate & Command Utama di bawah ini hanyalah contoh. Anda bisa menggunakan command – command lainnya dengan catatan harus mencakup keseluruhan spesifikasi dan terdokumentasi dengan baik.

5.1 Logika Inti Papan (core.pl)

Predicate	Penjelasan
<code>initial_board(-Board)</code>	Menghasilkan papan awal: list 7 kolom, tiap kolom 6 sel e.
<code>cell(+B,+Col,+Row,-Val)</code>	Mengambil nilai sel pada Kolom (1–7) dan Baris (1–6); gagal bila di luar batas.
<code>valid_move(+B,?Col)</code>	Benar bila kolom sah (1–7) dan belum penuh.
<code>drop(+B,+Col,+P,-NB)</code>	Menjatuhkan bidak P ke kolom, menghasilkan papan baru NB.
<code>drop_in_col(+Col,+P,-NC)</code>	Rekursif: mengisi sel e terbawah pada satu kolom.
<code>replace_nth(+L,+N,+X,-L2)</code>	Mengganti elemen ke-N sebuah list dengan X.
<code>won(+B,?P)</code>	Benar bila pemain P memiliki 4 bidak sejajar.
<code>four(+B,+P,+C,+R,+DC,+DR)</code>	Memeriksa 4 sel P mulai (C,R) searah vektor (DC,DR).
<code>board_full(+B)</code>	Benar bila tidak ada langkah sah tersisa (seri).
<code>terminal_like(+B)</code>	Benar bila ada yang menang ATAU papan penuh.

5.2 AI Inti: Negamax + Alpha-Beta + TT (ai.pl)

Predicate	Penjelasan
<code>best_move(+B,+P,+D,-Col)</code>	Langkah terbaik gaya <i>balanced</i> (kompatibel API lama).
<code>best_move_styled(+B,+P,+D,+Style,-Col)</code>	Langkah terbaik dengan gaya (personality) tertentu; me-reset statistik & TT di awal.
<code>negamax(+B,+P,+D,+A,+Bt,+Cfg,-V)</code>	Negamax + alpha-beta + TT + statistik. Cfg = <code>cfg(RootPlayer,Style)</code> .
<code>nm_loop/10</code>	Loop langkah dengan pemangkasan; mengembalikan flag Exact (yes/no) untuk keputusan caching TT.
<code>heuristic(+B,+P,+Style,-V)</code>	Evaluasi papan; bobot tiap jendela bergantung Style.
<code>ordered_moves(+B,-Ms)</code>	Urutan langkah <i>center-first</i> [4,3,5,2,6,1,7].
<code>tt_store/4, tt_entry/4</code>	Menyimpan/membaca cache Transposition Table (hanya nilai <i>exact</i>).
<code>reset_search_stats/0, bump_counter/1, get_counter/2</code>	Kelola statistik pencarian (nodes, alpha/beta cutoff, tt hit/miss).

<code>format_thousands(+N,-Atom)</code>	Format angka dengan pemisah ribuan (mis. 18245 -> 18,245).
<code>print_search_stats(+D,+Ms)</code>	Cetak blok statistik AI (Depth/Nodes/Cutoff/Time).

5.3 Analisis Langkah, Kesulitan & Personality

Predicate	Penjelasan
<code>immediate_win(+B,+P,-Col)</code>	Benar bila P bisa menang seketika dengan main di Col.
<code>opponent_threats(+B,+P,-Cs)</code>	Daftar kolom tempat lawan bisa menang giliran depan.
<code>top_moves(+B,+P,+D,-Sorted)</code>	Semua langkah terurut skor menurun, gaya <i>balanced</i> (untuk hint/heatmap).
<code>best_move_easy(+B,+P,+D,-Col)</code>	Mode Mudah: acak di antara langkah mendekati terbaik (margin skor).
<code>best_move_random(+B,+P,-Col)</code>	Personality Random: tetap ambil menang instan/blokir tunggal, selain itu acak murni.
<code>choose_move/5,</code> <code>choose_move_full/6</code>	Dispatcher tingkat kesulitan (+ gaya) -> kolom AI.
<code>book_or_search/5</code>	Cek Opening Book (via <code>move_log</code>) sebelum jatuh ke <code>best_move_styled</code> .
<code>style_mult(+Style,+Side,-Num,</code> <code>-Den)</code>	Tabel pengali bobot heuristik per gaya & sisi (self/opp).

5.4 Zobrist Hash & Principal Variation

Predicate	Penjelasan
<code>setup_zobrist/0</code>	Membangun tabel nilai acak Zobrist sekali saat program dimuat.
<code>zobrist_hash(+B,-Hash)</code>	Hash papan = XOR seluruh nilai sel terisi.
<code>fold_xor(+Vals,+Acc,-R)</code>	Rekursif melipat list nilai dengan <code>xor/2</code> .
<code>principal_variation(+B,+P,+D,</code> <code>-Eval,-PV)</code>	Garis terbaik (aproksimasi) + skor evaluasi akar.
<code>pv_continue/4</code>	Rekursif melanjutkan PV lewat pemanggilan <code>best_move</code> berulang.

5.5 Position Analysis, Explainable AI, Heat Map, Kompleksitas (features.pl)

Predicate	Penjelasan
<code>position_breakdown(+B,+P,-bd(</code> <code>...))</code>	Rincian: Center Control, Threat, Connect-2, Connect-3, Opponent Threat, Total.
<code>center_control/3,</code> <code>count_open_windows/4</code>	Komponen breakdown: kontrol tengah & jumlah jendela terbuka.
<code>explain_move(+B,+P,+Col,-Reasons)</code>	Kumpulan alasan (center/blokir/connect-3/fork) untuk sebuah langkah.
<code>print_explanation/4</code>	Cetak alasan + Evaluation Score (dipakai perintah <code>explain</code>).

<code>print_heatmap/3</code>	Cetak evaluasi seluruh 7 kolom, kolom terbaik ditandai *.
<code>complexity_analysis(+B,+D)</code>	Branching factor, estimasi node (BF^D), node sungguhan, efisiensi pruning.

5.6 Search Tree Visualization, Benchmark, Tournament, Puzzle

Predicate	Penjelasan
<code>trace_tree(+B,+P)</code>	Visualisasi pohon 2-ply berbasis alpha-beta NYATA (bukan simulasi).
<code>trace_root/7,</code> <code>trace_level2/6,</code> <code>trace_level2_loop/10</code>	Pencari & pencetak pohon per level, terpisah dari hot-path negamax.
<code>run_benchmark(+MaxDepth)</code>	Tabel Depth/Nodes/Time untuk kedalaman 1..MaxDepth.
<code>run_tournament(+DA,+DB,+N)</code>	N pertandingan AI-vs-AI; sisi ditukar tiap match genap; tally akhir.
<code>play_one_match(+DA,+DB,-</code> <code>Result)</code>	Satu pertandingan penuh, <i>failure-driven</i> , memakai <code>best_move_easy</code> untuk variasi.
<code>setup_puzzles/0,</code> <code>puzzle_fact/4</code>	Bank 5 puzzle (papan, giliran, deskripsi).
<code>check_puzzle_answer/4,</code> <code>puzzle_solution_hint/3</code>	Verifikasi programatik (<code>immediate_win</code> / <code>opponent_threats</code>), bukan hardcode.

5.7 Antarmuka, Undo/Redo, Replay, File (io.pl)

Predicate	Penjelasan
<code>play/0,</code> <code>menu_loop/0,</code> <code>show_menu/0,</code> <code>do_menu/1</code>	Entry point & menu 11-opsi, <i>failure-driven</i> .
<code>start_game/3,</code> <code>game_loop/0,</code> <code>take_turn/4</code>	Menyiapkan & menjalankan satu permainan; satu giliran manusia/AI.
<code>human_move/5,</code> <code>dispatch/6</code>	Membaca & menangani seluruh perintah dalam permainan.
<code>do_undo/0,</code> <code>do_redo/0,</code> <code>push_undo/1,</code> <code>push_redo/1</code>	Undo/redo tak terbatas berbasis snapshot 4-tuple (papan,giliran,kedalaman,move_log).
<code>move_is_risky(+B,+P,+Col)</code>	Smart Hint: benar bila langkah membuka kemenangan lawan giliran depan.
<code>adaptive_suggestion/0</code>	Saran penyesuaian kesulitan dari rasio menang/kalah tersimpan.
<code>setup_replay/1,</code> <code>replay_show/1,</code> <code>replay_next/0,</code> <code>replay_prev/0,</code> <code>replay_jump/1</code>	Replay interaktif: navigasi bebas atas list papan hasil pra-hitung.

export_replay_ascii/1, export_replay_json/1	Ekspor riwayat ke berkas ASCII animation / JSON.
save_game/load_game, save_log/load_log, load_stats/save_stats	Baca/tulis state, riwayat langkah, dan statistik ke berkas.

Perintah saat bermain (semua diakhiri titik): 1–7 jatuhkan bidak | hint | analysis | heatmap | tree | complexity | pv | zobrist | explain | undo / redo | board | save('g.txt') | savelog('l.txt') | help | quit.

Perintah dalam Replay Interaktif: next | prev | jump(N) | export(ascii,'f.txt') | export(json,'f.json') | quit.

6. Algoritma AI

6.1 Negamax + Alpha-Beta + Transposition Table

AI menggunakan formulasi **negamax** — varian ringkas minimax yang memanfaatkan sifat *skor lawan = negatif skor kita* — dipadu **alpha-beta pruning** dan **Transposition Table**. Fungsi evaluasi memberi skor papan berdasarkan seluruh kemungkinan ‘jendela’ empat sel: jendela berisi 1/2/3 bidak sendiri (tanpa bidak lawan) bernilai +1/+10/+50 (dikalikan bobot gaya bermain (lihat 6.3)), dan sebaliknya untuk lawan. Kemenangan diberi skala *mate-distance* (1.000.000 + sisa kedalaman) sehingga **menang lebih cepat selalu lebih diutamakan** daripada menang yang lebih lambat.

```
negamax(Board, Player, Depth, Alpha, Beta, Cfg, Value) :-
    bump_counter(nodes),
    ( tt_entry(Board, Player, Depth, Value) ->
        bump_counter(tt_hit)
    ; bump_counter(tt_miss),
        compute_negamax(Board, Player, Depth, Alpha, Beta, Cfg, Value)
    ).
```

Transposition Table **hanya menyimpan nilai EXACT** — hasil nm_loop yang selesai penuh tanpa cutoff (ditandai flag Exact = yes). Nilai hasil cutoff hanyalah *bound* (batas bawah/atas), bukan nilai pasti, sehingga sengaja TIDAK di-cache — menghindari kompleksitas flag EXACT/LOWERBOUND/UPPERBOUND yang lazim pada engine catur, dengan trade-off cache-hit sedikit lebih jarang namun selalu aman dipakai ulang tanpa syarat tambahan.

6.2 Klasifikasi Alpha Cutoff vs Beta Cutoff

Karena negamax menyatukan “maximize” di setiap node (selalu dari sudut pandang pemain yang jalan), klasifikasi cutoff memakai kesetaraan negamax<->minimax standar: negamax(n) = max(n) bila n adalah node MAX pada pohon klasik (giliran RootPlayer), dan negamax(n) = -min(n) bila n node MIN (giliran lawan root). Maka pada implementasi ini:

- Cutoff pada node dengan Player == RootPlayer — setara node MAX klasik — dihitung sebagai **Beta Cutoff**.
- Cutoff pada node dengan Player = lawan root — setara node MIN klasik — dihitung sebagai **Alpha Cutoff**.

```
( Alpha >= Beta ->
    Value = Acc1, Exact = no,
    Cfg = cfg(RootPlayer, _),
    ( Player == RootPlayer -> bump_counter(beta_cut)
    ; bump_counter(alpha_cut) ), !
; nm_loop(Rest, ..., Value, Exact) ).
```

6.3 AI Personality (Gaya Bermain)

Empat gaya memengaruhi bobot heuristik pada sisi “menyerang” (bidak sendiri) vs “bertahan” (blokir bidak lawan):

Predicate	Penjelasan
Balanced	Bobot standar kedua sisi ($\times 1 / \times 1$).
Aggressive	Bobot menyerang $\times 1,5$; bertahan $\times 1$ — condong membangun ancaman sendiri.
Defensive	Bobot menyerang $\times 1$; bertahan $\times 1,5$ — condong menutup ancaman lawan lebih dini.
Random	Tidak memanggil minimax sama sekali — namun tetap mengambil kemenangan instan / memblokir ancaman tunggal sebelum jatuh ke pilihan acak murni, agar tetap kompeten secara taktis.

Seluruh gaya (termasuk Random) diverifikasi tetap mengambil kemenangan instan dan memblokir ancaman tunggal pada posisi uji taktis.

6.4 Opening Book & Tingkat Kesulitan Expert

Sebelum memanggil minimax, AI mengecek `book_move_fact(MoveLog, Col)` menggunakan riwayat langkah (`move_log`) sebagai kunci. Buku ini berisi ~ 8 entri representatif untuk 1–2 langkah pembuka (*ilustratif*, bukan basis data solved-game lengkap — lihat Bagian 9). Tingkat **Expert** = kedalaman 5 + Opening Book + evaluasi bergaya; kedalaman TIDAK dinaikkan ke 6 karena terbukti overflow (lihat Bagian 9).

6.5 Kinerja Terukur

Predicate	Penjelasan
Depth 4, tanpa instrumentasi	$\sim 167\text{--}182$ ms, kolom 4 (pembuka optimal).
Depth 5, tanpa instrumentasi	$\sim 530\text{--}540$ ms.
Depth 5, DENGAN statistik+TT	~ 887 ms — overhead assert/retract (lihat Bagian 9).
Depth 6 (papan awal)	Global stack overflow. dipertahankan sebagai batas atas Custom, TIDAK dipakai Expert.

7. Pemetaan Enam Konsep Wajib

Konsep	Implementasi dalam Kode
Rekurens	<code>negamax/7</code> , <code>nm_loop/10</code> , <code>drop_in_col/3</code> , <code>replace_nth/4</code> , <code>four/6</code> , <code>principal_variation/pv_continue</code> , <code>build_replay_boards</code> , <code>fold_xor</code> , rekursi pencetakan papan & search tree.
List	Papan (list 7 kolom $\times$ 6 sel), list ‘jendela’ 4-sel, pasangan Skor-Kolom, <code>move_log</code> , <code>undo_stack/redo_stack</code> (snapshot 4-tuple), list Boards pada replay interaktif.
Cut	Commit hasil <code>best_move_styled/5</code> ; <i>beta cutoff</i> pada <code>nm_loop/10</code> ; klausa <code>dispatch/6</code> , <code>set_difficulty/2</code> , <code>set_style_choice/2</code> , <code>check_puzzle_answer/4</code> .

Fail	Penolakan langkah ilegal di dispatch/6; undo/redo mem-fail-kan giliran untuk mereklaim global stack; play_one_match/3 (Tournament) <i>failure-driven</i> penuh.
Loop	Loop <i>failure-driven</i> repeat...fail (game loop, menu, Tournament) dan between...fail (papan, search tree, ekspor ASCII).
File Processing	save_game/load_game, save_log/load_log, load_stats/save_stats, export_replay_ascii/export_replay_json (open/read/write/close).

8. Contoh Query dan Output

8.1 Menjalankan Program

```
$ gprolog --consult-file connect4.pl
=====
CONNECT FOUR - GNU Prolog + AI
minimax + alpha-beta + fitur lanjutan
=====
```

```
Menu Utama:
1. Kamu (X) vs AI (O)
2. AI (X) vs Kamu (O) [AI jalan duluan]
3. Dua Pemain (X vs O)
4. Demo AI vs AI
5. Muat permainan tersimpan
6. Replay interaktif riwayat
7. Lihat statistik
8. Benchmark AI
9. AI vs AI Tournament
10. Puzzle Mode
11. Keluar
Pilih (akhiri titik) : 1.
```

8.2 AI Mengambil Kemenangan Instan & Menutup Ancaman

```
%% X hampir menang (tiga bidak sejajar kolom 1-3, butuh kolom 4)
?- best_move(Board, x, 5, Col). %% giliran X
Col = 4 %% X mengambil kemenangan
```

```
?- best_move(Board, o, 5, Col). %% giliran O
Col = 4 %% O menutup ancaman X
```

```
%% O punya 3 bidak vertikal di kolom 2
?- best_move(Board, o, 5, Col).
Col = 2 %% O menuntaskan kemenangan vertikal
```

8.3 AI Statistics & Transposition Table

AI (O) berpikir...

```
Depth      : 5
Nodes      : 1,885
Alpha Cutoff : 54
Beta Cutoff  : 418
Time       : 0.89 sec
```

AI (O) menjatuhkan bidak di kolom 4

Konsistensi terverifikasi: tt_hit + tt_miss = nodes pada setiap pencarian (mis. 377 + 1.508 = 1.885 pada pengujian internal).

8.4 Search Tree Visualization (tree.)

```

Depth 0
|-- Column 4
|   |-- Score = 3
|   |-- Score = -2
|   |-- ... (7 anak dieksplorasi penuh - kolom pertama menetapkan alpha)
|-- Column 3
|   |-- Score = 2
|   |-- Score = 3
|       (Beta Cutoff - sisa kolom dipangkas)
|-- Column 5
|   |-- Score = 2
|   |-- Score = 0
|   |-- Score = 3
|       (Beta Cutoff - sisa kolom dipangkas)
`-- Column 7
    |-- Score = 4
        (Beta Cutoff - sisa kolom dipangkas)

```

Pohon ini dihasilkan oleh pencari alpha-beta 2-ply yang TERPISAH dari hot-path negamax utama (agar tak mengganggu kode inti yang sudah teruji), namun tetap menjalankan alpha-beta sungguhan — label “Beta Cutoff” yang tampil adalah cutoff nyata, bukan simulasi.

8.5 Explainable AI (explain.) & Position Analysis (analysis.)

```

AI memilih kolom 4

Alasan:
- Mengontrol center
- Membuka lebih dari satu peluang kemenangan (fork)

Evaluation Score : +1000004

Position Analysis

Center Control   : +3
Threat           : +25
Connect-2        : 1
Connect-3        : 1
Opponent Threat  : +0

Total Score      : +67

```

8.6 Heat Map (heatmap.) & Analisis Kompleksitas (complexity.)

```

Heat Map Evaluasi Kolom

Column 1 : +19
Column 2 : +28
Column 3 : +44
Column 4 : +1000004 *
Column 5 : +15
Column 6 : +9
Column 7 : +9

Analisis Kompleksitas

Current Branching Factor : 7
Estimated Nodes          : 2,401
Visited Nodes            : 213
Pruning Efficiency       : 92%

```

8.7 Zobrist Hash (zobrist.) & Principal Variation (pv.)

```

Zobrist Hash posisi saat ini: 27940452183444632

Best Line

```

```
X : 4
O : 4
X : 4
O : 3
Evaluation : -4
```

8.8 Hint, Peringatan Ancaman & Smart Hint

```
!! Awas: lawan mengancam menang di kolom [4]

X, pilih kolom 1-7 ... : hint.
>> Analisis AI (3 kolom terbatas):
    kolom 4 : skor 1000005 (MENANG!)
    kolom 6 : skor 7
    kolom 5 : skor 5
>> Rekomendasi: kolom 4

O, pilih kolom 1-7 ... : 5.

Peringatan

Jika memilih kolom 5, lawan bisa menang giliran berikutnya.

Lanjutkan? (y/n) : n.
>> Dibatalkan, pilih kolom lain.
```

8.9 Undo, Redo & Statistik

```
X, pilih kolom 1-7 ... : undo.
(kembali ke giliran manusia sebelumnya - langkah pemain
dan balasan AI dibatalkan sekaligus)

X, pilih kolom 1-7 ... : redo.
(maju kembali ke posisi yang tadi di-undo, move_log utuh)

===== STATISTIK =====
Menang X : 2
Menang O : 1
Seri      : 0
Total     : 3
=====
```

8.10 Replay Interaktif & Export

```
Nama file move-log (mis. 'log.txt'.) : 'rlog.txt'.

Posisi 0 / 7
|.....|
... (papan kosong) ...
> next.

Posisi 1 / 7
... (papan setelah 1 langkah) ...
(langkah ke-1: kolom 4)
> jump(7).
... (posisi akhir) ...
> export(json,'replay.json').
>> Diekspor (JSON) ke replay.json

%% isi replay.json:
{"moves":[4,3,4,3,4,3,4]}
```

8.11 Benchmark AI & AI vs AI Tournament

Depth	Nodes	Time
1	7	0.00 s
2	56	0.02 s
3	180	0.05 s
4	622	0.20 s

Tournament

AI Depth 2

VS

AI Depth 4

Match : 8

Depth 2 Win : 2

Depth 4 Win : 5

Draw : 1

Catatan kinerja: durasi per-match naik tajam bersama kedalaman (mis. ~1 detik/match pada 2 vs 2, namun ~15 detik/match pada 2 vs 4) — menu memberi peringatan bila kedalaman > 3 dikombinasikan dengan jumlah match > 10.

8.12 Puzzle Mode

```
=== Puzzle Mode ===
```

```
Puzzle 1
```

```
Puzzle 2
```

```
Puzzle 3
```

```
Puzzle 4
```

```
Puzzle 5
```

```
Pilih nomor puzzle (akhiri titik, 0 utk kembali) : 1.
```

```
Puzzle 1
```

```
Giliran X. Temukan langkah kemenangan.
```

```
... (papan) ...
```

```
Kolom pilihan (X, akhiri titik) : 4.
```

```
Correct!
```

```
Winning Move Found.
```

8.13 Menyimpan & Memuat Permainan

```
X, pilih kolom 1-7 ... : save('game.txt').
```

```
>> Disimpan ke game.txt
```

```
%% isi berkas game.txt (satu term Prolog):
```

```
game([[e,e,e,e,e,e],[o,e,e,e,e,e],[x,o,e,e,e,e],  
      [x,e,e,e,e,e],[e,e,e,e,e,e],[e,e,e,e,e,e],  
      [e,e,e,e,e,e]], x, 4).
```

```
%% memuat kembali lewat menu (5):
```

```
Dimuat. Giliran: x
```

9. Catatan Implementasi (Trade-off yang Disengaja)

Bagian ini mendokumentasikan keputusan desain, keterbatasan yang disadari, dan kuirk GNU Prolog yang ditemukan selama pengembangan ditulis, termasuk temuan yang kurang menguntungkan, sebagai bagian dari pertanggungjawaban teknis. Bagian ini bisa ditaruh nantinya pada spesifikasi final, atau cuman sebatas bahan pertimbangan nantinya.

9.1 Failure-Driven Game Loop (Global Stack)

GNU Prolog tidak melakukan *garbage collection* global stack secara otomatis. Pada rekursi maju yang panjang (permainan berpuluh langkah), term yang dialokasikan minimax tidak terbebaskan sehingga berpotensi menimbulkan *global stack overflow*. Solusinya: game loop, mode Demo, dan AI vs AI Tournament seluruhnya dirancang **failure-driven**, state disimpan pada dynamic predicate, tiap langkah diproses lalu sengaja di-fail-kan agar Prolog backtrack dan mereklaim global stack sebelum langkah berikutnya.

9.2 GNU Prolog Tanpa Bignum atau dengan kata lain “Overflow Diam-diam”

Ditemukan saat menguji Zobrist Hash: GNU Prolog pada lingkungan ini **tidak mendukung bignum**. Ekspresi sederhana 2^{62} menghasilkan 0 tanpa peringatan atau error apa pun. Pengujian manual menunjukkan batas amannya di sekitar 2^{59} (representasi ~61-bit bertanda, konsisten dengan skema tag pointer internal). Desain awal tabel Zobrist (kombinasi *shift* 40-bit dan 20-bit) melewati batas ini dan menghasilkan hash yang salah/tidak terduga; diperbaiki dengan kombinasi *shift* yang lebih kecil (~58-bit efektif, aman). Ini menjadi alasan tambahan mengapa Transposition Table proyek ini TIDAK memakai Zobrist hash sebagai kunci, representasi papan (list) sebagai kunci sama sekali tidak berisiko overflow.

9.3 Overhead Instrumentasi vs Manfaat Transposition Table

Pengukuran langsung: pencarian depth-5 pada papan awal **tanpa** statistik/TT ~ 537 ms; **dengan** statistik/TT ~ 887 ms — naik ~65%. Penyebabnya: assert/retract per node (untuk counter dan cache) relatif mahal di GNU Prolog, melebihi penghematan yang didapat dari cache-hit TT pada skala pohon pencarian proyek ini (ribuan node). Tetap sepenuhnya *playable* (di bawah 1 detik); dilaporkan apa adanya sebagai temuan jujur karena fitur ini tetap bernilai untuk tujuan edukasi/demonstrasi konsep, bukan untuk mengejar kecepatan mentah.

9.4 Transposition Table: Hanya Nilai Exact

TT hanya menyimpan nilai dari `nm_loop` yang selesai penuh tanpa cutoff (flag Exact = yes). Nilai hasil cutoff adalah *bound*, bukan nilai pasti sengaja tidak di-cache untuk menghindari kompleksitas skema EXACT/LOWERBOUND/UPPERBOUND. Tabel dikosongkan di **awal setiap langkah AI** (bukan sepanjang permainan) karena database GNU Prolog tidak ber-GC otomatis; menyimpan lintas-langkah berisiko membengkak tanpa batas pada permainan panjang.

9.5 Opening Book

Buku pembuka berisi ~8 entri representatif untuk 1–2 langkah awal, **bukan** basis data solved-game Connect Four yang lengkap (basis data semacam itu memerlukan jutaan posisi hasil perfect-play search dan berada di luar cakupan tugas ini). Tujuannya mendemonstrasikan MEKANISME — AI mengecek buku dulu sebelum memanggil minimax — bukan mengklaim permainan sempurna sejak langkah pertama.

9.6 AI vs AI Tournament

Versi awal Tournament memakai `best_move` yang sepenuhnya deterministik — akibatnya SEMUA N match antar konfigurasi yang sama menghasilkan permainan identik bit-demi-bit (selalu seri, tidak

informatif). Diperbaiki dua langkah: **(a)** memakai `best_move_easy` (mendekati-terbaik, dengan margin acak) sehingga tiap game punya variasi nyata; **(b)** menukar sisi X/O pada match bernomor genap (`swap_result/2`) agar tak ada bias keuntungan jalan-duluan. Hasilnya: tally kemenangan yang bervariasi dan mencerminkan kekuatan relatif tiap kedalaman.

9.7 Principal Variation merupakan Aproksimasi, Bukan Ekstraksi Murni

PV dibangun lewat pemanggilan `best_move` berulang pada kedalaman menurun (`pv_continue/4`), bukan diekstraksi langsung dari satu pencarian negamax — itu memerlukan menyimpan rantai langkah terbaik di tiap node, perubahan yang lebih invasif ke inti pencarian yang sudah teruji ketat. Pada gaya *balanced* (deterministik) garis yang dihasilkan pada praktiknya konsisten.

9.8 Puzzle Mode: Dua dari Lima Puzzle Awal Salah Konstruksi

Karena jawaban diverifikasi **programatik** (kolom benar = memberi kemenangan instan ATAU memblokir ancaman instan lawan, dicek via `immediate_win/opponent_threats` — bukan `hardcode`), kesalahan konstruksi papan langsung diketahui saat pengujian: Puzzle #2 awalnya menumpuk bidak X-O berselang-seling pada satu kolom sehingga tidak pernah benar-benar membentuk ancaman tiga-beruntun (diperbaiki: tiga X murni bertumpuk). Puzzle #5 awalnya menempatkan O pada kolom yang membuka ancaman GANDA (dua kolom berbeda sama-sama menuntaskan kemenangan, tak bisa diblokir satu langkah), yang akan membuat pengecekan “blokir benar” menyesatkan (diperbaiki: posisi ditata ulang agar hanya satu kolom yang benar-benar menuntaskan ancaman). Kelima puzzle diverifikasi ulang lolos setelah perbaikan.

9.9 Kuirk Sintaks & Builtin GNU Prolog yang Ditemukan

Predicate	Penjelasan
<code>Literal % dalam format/2</code>	Melempar <code>domain_error(non_empty_list,[])</code> . Solusi: <code>write('%')</code> terpisah dari pemanggilan <code>format</code> .
<code>format(atom(A), mt, Args)</code> F	Tidak didukung (<i>unsupported</i>). Solusi: cetak langsung ke stream, bukan bangun atom berformat.
<code>xor</code> sebagai operator infix	Bukan operator baku — harus dipanggil sebagai fungsi <code>xor(A,B)</code> , bukan <code>A xor B</code> .
<code>**</code> vs <code>^</code>	<code>**</code> SELALU mengembalikan float meski kedua operand integer (<code>3**4 = 81.0</code>); <code>^</code> mengembalikan integer bila operand integer (<code>3^4 = 81</code>).
<code><<</code> (shift kiri)	Operator infix valid dan berfungsi normal — bukan sumber masalah, hanya perlu diwaspadai batas overflow (lihat 9.2).
<code>set_seed/1</code>	Nama predicate builtin GNU Prolog — PRNG kustom proyek ini diberi nama <code>rng_set/1</code> untuk menghindari konflik “native code procedure cannot be redefined”.
<code>double_quotes flag</code>	Default GNU Prolog memperlakukan string bertanda kutip-dua sebagai list kode karakter, bukan atom — <code>:- set_prolog_flag(double_quotes, atom).</code> wajib di awal berkas.

9.10 Batas Kedalaman: Expert Tetap 5, Bukan 6

Kedalaman 6 dari papan awal diuji ULANG setelah penambahan Transposition Table (dengan harapan cache mengurangi puncak penggunaan stack) — hasilnya tetap *global stack overflow*. Karena

tingkat Expert dipertahankan pada kedalaman 5 (sama seperti Sulit), diperkuat Opening Book dan evaluasi bergaya, alih-alih menaikkan kedalaman ke titik yang terbukti tidak stabil pada stack bawaan.